Relatório Final de Testes: Agenda Central

1. Introdução ao sistema desenvolvido

O sistema **Agenda Central** é uma aplicação Java baseada em interface de linha de comando (CLI) voltada para o gerenciamento pessoal de contatos. A arquitetura foi estruturada utilizando o padrão DAO para separar a regra de negócio da camada de persistência em um banco de dados MySQL. As principais funcionalidades incluem o registro e autenticação de usuários, garantindo que o escopo de dados seja isolado, permitindo que cada usuário realize operações de CRUD (Criar, Ler, Atualizar e Excluir) exclusivamente em sua própria lista de contatos.

2. Planejamento de testes

Nesta etapa, a estratégia de testes adotada contemplou **testes funcionais de caixa-preta** executados manualmente através do console da aplicação. O objetivo principal é validar os fluxos de ponta a ponta na classe principal, simulando o comportamento real de um usuário final sem interagir diretamente com o código-fonte.

- Validação das regras de autenticação (cadastro de conta, sucesso e falha no login).
- Verificação rigorosa do isolamento de dados entre múltiplas contas (garantir que as consultas SQL protegem a privacidade da agenda de cada usuário).

3. Descrição dos casos de teste

CT01: Cadastro de Usuário e Autenticação Inválida

- **Objetivo:** Verificar a criação de um novo usuário e a rejeição de login com credenciais incompletas ou incorretas.
- **Ação:** Cadastrar um usuário. Em seguida, tentar realizar o login deixando a senha em branco ou digitando uma senha errada.
- **Resultado Esperado:** O sistema deve exibir a mensagem "Usuário criado!" no primeiro passo. Na falha de login, deve exibir "Erro: Credenciais inválidas." e retornar ao menu inicial sem conceder acesso.

CT02: Validação de Isolamento de Dados (Múltiplos Usuários)

- **Objetivo:** Garantir que a agenda de contatos seja privada e restrita apenas ao usuário autenticado no momento.

- **Ação:** 1. Cadastrar duas contas distintas. 2. Logar com a primeira conta, adicionar um contato na agenda e realizar o logout. 3. Logar com a segunda conta e solicitar a listagem de contatos.
- **Resultado Esperado:** Ao listar os contatos com a segunda conta, o contato cadastrado pela primeira conta não deve aparecer, resultando na exibição de uma lista vazia.

CT03: Validação de Limite de Caracteres e Tratamento de Exceção (Falha Encontrada)

- **Objetivo:** Verificar o comportamento do sistema ao receber uma entrada de dados (telefone) com tamanho maior do que o limite suportado pela coluna no banco de dados.
- **Ação:** Logar no sistema, selecionar a opção de adicionar "Novo Contato", preencher o nome e inserir uma string propositalmente longa no campo telefone (ex: `12345678912345678912 3`). Após salvar, realizar a listagem da agenda.
- **Resultado Esperado:** O sistema deve validar o limite de caracteres na entrada ou capturar o erro do banco de dados de forma amigável, informando ao usuário que a operação falhou. A mensagem de sucesso não deve ser exibida se o dado não for persistido.
- **Resultado Obtido (Bug):** A aplicação falhou ao tratar o erro adequadamente. O sistema exibiu a mensagem de sucesso falso ("Contato salvo!") seguida pelo vazamento de uma exceção técnica do banco de dados no console (`MysqlDataTruncation: Data too long for column 'telefone'`). A listagem posterior comprovou que o contato não foi armazenado, quebrando a confiabilidade do retorno do sistema.

4. Resultados das execuções e evidências

Abaixo estão as evidências demonstrando a execução dos testes de caixa-preta descritos anteriormente, realizados diretamente no console da aplicação.

Evidência 1 - Referente ao CT01:

```
--- AGENDA CENTRAL ---
1. Login | 2. Cadastro | 0. Sair
Escolha: 2
Novo Login: dimitri
Nova Senha: 123456
Usuário criado!

--- AGENDA CENTRAL ---
1. Login | 2. Cadastro | 0. Sair
Escolha: 1
Login: dimitri
Senha:
Erro: Credenciais inválidas.
```

Análise do Resultado: O cadastro do usuário "dimitri" foi processado com sucesso. Logo abaixo, a tentativa de login do mesmo usuário com falha na inserção da senha acionou corretamente o bloqueio de segurança do sistema, exibindo o erro de credenciais.

Evidência 2 - Referente ao CT02 (Parte 1):

```
--- AGENDA CENTRAL ---
1. Login | 2. Cadastro | 0. Sair
Escolha: 2
Novo Login: enzo
Nova Senha: 123456
Usuário criado!

--- AGENDA CENTRAL ---
1. Login | 2. Cadastro | 0. Sair
Escolha: 2
Novo Login: guilherme
Nova Senha: 123456
Usuário criado!

--- AGENDA CENTRAL ---
1. Login | 2. Cadastro | 0. Sair
Escolha: 1
Login: guilherme
Senha: 123456

Agenda de: guilherme
1. Novo Contato | 2. Listar | 3. Editar | 4. Excluir | 0. Logout
Opção: 1
Nome: pai
Telefone: 61981818282
Contato salvo!

Agenda de: guilherme
1. Novo Contato | 2. Listar | 3. Editar | 4. Excluir | 0. Logout
Opção: 0

--- AGENDA CENTRAL ---
1. Login | 2. Cadastro | 0. Sair
Escolha: 1
Login: enzo
Senha: 123456
```

Análise do Resultado: Demonstração do cadastro de duas contas. A conta "guilherme" realiza o acesso e consegue incluir o contato "pai" em sua agenda com sucesso, realizando o logout em seguida.

Evidência 3 - Referente ao CT02 (Parte 2):

```
Agenda de: enzo
1. Novo Contato | 2. Listar | 3. Editar | 4. Excluir | 0. Logout
Opção: 2

--- Seus Contatos ---
```

Análise do Resultado: Comprova o sucesso absoluto do isolamento de dados do banco. Ao acessar a conta, a listagem da agenda retorna vazia, comprovando que o contato inserido anteriormente está atrelado exclusivamente à chave estrangeira (`id_usuario`) da primeira conta.

Adicione na seção "4. Resultados das execuções e evidências":

Evidência 4 - Referente ao CT03 (Falha de Truncamento de Dados):

```
--- AGENDA CENTRAL ---
1. Login | 2. Cadastro | 0. Sair
Escolha: 1
Login: dimitri
Senha: 123456

Agenda de: dimitri
1. Novo Contato | 2. Listar | 3. Editar | 4. Excluir | 0. Logout
Opção: 1
Nome: aldair
Telefone: 123456789123456789123
Contato salvo!
com.mysql.cj.jdbc.exceptions.MysqlDataTruncation Create breakpoint : Data truncation: Data too long for column 'telefone' at row 1
    at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:104)
    at com.mysql.cj.jdbc.ClientPreparedStatement.executeInternal(ClientPreparedStatement.java:912)
    at com.mysql.cj.jdbc.ClientPreparedStatement.execute(ClientPreparedStatement.java:354)
    at br.com.agenda.dao.ContatoDAO.save(ContatoDAO.java:18)
    at br.com.agenda.aplicacao.Main.menuAgenda(Main.java:55)
    at br.com.agenda.aplicacao.Main.main(Main.java:27)

Agenda de: dimitri
1. Novo Contato | 2. Listar | 3. Editar | 4. Excluir | 0. Logout
Opção: 2

--- Seus Contatos ---
```

Análise do Resultado (Defeito): A imagem comprova uma falha arquitetural no tratamento de erros da aplicação. A classe `ContatoDAO.java` captura a exceção de violação de limite de caracteres do MySQL (linha 18) e apenas imprime a pilha de rastreamento no console (`e.printStackTrace()`), não repassando o erro para a classe `Main`. Como consequência, a `Main` executa a linha que imprime "Contato salvo!" de forma enganosa. O passo seguinte (Opção 2 - Listar) retorna uma lista vazia, confirmando a não-persistência dos dados e a falha de comunicação entre a camada DAO e a interface do usuário.

5. Conclusões e lições aprendidas

A execução dos testes de caixa-preta evidenciou o correto funcionamento dos mecanismos de controle de acesso da aplicação. O **CT01** demonstrou a eficácia da validação na camada inicial de autenticação, impedindo acessos não autorizados. Já o **CT02** provou na prática que o uso da chave estrangeira no banco de dados, associado às consultas parametrizadas (uso do `PreparedStatement`), garante o particionamento seguro das informações. Uma lição importante extraída desta execução é que realizar testes simulando múltiplos perfis simultâneos é uma etapa indispensável para garantir que não existam vazamentos de dados entre sessões diferentes. Por fim, a execução do **CT03** revelou uma vulnerabilidade significativa na forma como o sistema lida com o tratamento de exceções (Exception Handling). A lição aprendida é que não basta a camada de persistência (DAO) apenas capturar as exceções SQL; ela deve propagar esses erros (lançar a exceção ou retornar um valor booleano falso) para a camada de apresentação (`Main.java`). Isso é crucial para que o usuário receba um feedback real e condizente com o estado do banco de dados, evitando mensagens de falso positivo como a observada neste teste de caixa-preta.

6. Link para o repositório no GitHub

https://github.com/Dimitri-Vieira/Projeto-Teste-Software-Agenda/blob/main/src/main/java/br/com/agenda/model/Usuario.java